REETRAK AUTOMATION PROJECT GUIDE

1. PROJECT OVERVIEW

This project is a UI automation framework for the Reetrak application. It uses:

* Java 21 for implementation
* Maven for dependency management and execution
* Selenium WebDriver for browser automation
* TestNG for test execution and assertions
* Page Object Model (POM) for maintainable page interactions
* WebDriverManager for automatic browser driver setup
* A custom HTML reporter for execution summaries

In simple terms, this project opens a browser, performs actions on the Reetrak login page, validates expected behavior, captures screenshots, and generates a test report after execution.

2. WHAT THIS PROJECT IS FOR

Use this project when you want to:

* verify login and logout flows automatically
* validate negative scenarios like blank credentials, invalid users, deactivated users, and locked users
* run repeated regression checks without manual browser testing
* keep UI test logic organized using reusable page classes
* generate execution evidence with screenshots and HTML summary reports

This framework is best suited for smoke testing, regression testing, and feature validation of web-based login-related workflows.

3. WHY THIS FRAMEWORK DESIGN IS USED

JAVA

Java is widely used for enterprise test automation. It works well with Selenium, TestNG, Maven, reporting libraries, and CI tools.

MAVEN

Maven makes dependency management predictable. It also standardizes how the project is compiled and how tests are run.

SELENIUM WEBDRIVER

Selenium controls real browsers like Chrome, Edge, and Firefox. That makes it suitable for end-to-end UI testing.

TESTNG

TestNG provides:

* test annotations such as ‘@Test‘, ‘@BeforeMethod‘, and ‘@AfterMethod‘
* assertions
* execution ordering using priorities
* suite control through ‘testng.xml‘
* listener support for custom reports

PAGE OBJECT MODEL

POM helps separate:

* test logic: what should be validated
* page logic: how elements are located and interacted with

This keeps tests cleaner and makes locator changes easier to maintain.

4. CURRENT PROJECT STRUCTURE

text

```
Reetrak Automation/
|-- pom.xml
|-- testng.xml
|-- src/
|   |-- main/java/com/reetrak/automation/
|   |   |-- factory/
|   |   |   -- DriverFactory.java
|   |   |-- pages/
|   |   |   |-- BasePage.java
|   |   |   -- LoginPage.java
|   |   -- utilities/
|   |       |-- ConfigReader.java
|   |       |-- ScreenshotUtils.java
|   |       -- WaitUtils.java
|   -- test/
|       |-- java/com/reetrak/automation/
|       |   |-- base/
|       |   |   -- BaseTest.java
|       |   |-- reports/
|       |   |   -- HtmlSummaryReporter.java
|       |   -- tests/
```

```
|       |       -- LoginLogoutTest.java
|       -- resources/
|            -- config.properties
|-- test-output/
|   -- html-summary-report.html
-- target/
```

5. HOW THE FRAMEWORK WORKS

STEP 1: CONFIGURATION IS LOADED

ConfigReader.java loads values from config.properties. It can also accept JVM system properties, which means runtime values can override file-based values.

Examples:

* ‘browser=chrome‘
* ‘base.url=http://localhost:8080/...‘
* ‘testdata.username=Tadmin‘

STEP 2: TEST SETUP STARTS BEFORE EVERY TEST METHOD

BaseTest.java runs setUp() before each test.

It does the following:

* reads ‘browser‘
* reads ‘base.url‘
* reads all ‘testdata.*‘ values
* initializes a browser through ‘DriverFactory‘
* stores the active ‘WebDriver‘

This means each test method starts with a fresh browser session.

STEP 3: BROWSER IS CREATED

DriverFactory.java is responsible for browser setup.

It supports:

* Chrome
* Firefox
* Edge

Important details:

* WebDriverManager automatically resolves the driver binary
* browser window is maximized
* implicit wait is disabled
* page load timeout is set to 30 seconds
* ‘ThreadLocal<WebDriver>‘ is used, which helps if the project later moves toward parallel execution

STEP 4: PAGE OBJECT INTERACTS WITH THE APPLICATION

LoginPage.java contains locators and reusable methods such as:

* ‘open(baseUrl)‘
* ‘enterUsername()‘
* ‘enterPassword()‘
* ‘clickLogin()‘
* ‘loginAs()‘
* ‘logoutAs()‘
* getters for validation messages

This class inherits from BasePage.java, which centralizes common actions:

* typing
* clicking
* reading text

BasePage.java itself relies on WaitUtils.java so interactions wait for visibility or clickability before acting.

STEP 5: TEST CLASS VALIDATES BEHAVIOR

LoginLogoutTest.java contains the actual test scenarios.

Current tests cover:

1. blank username and password
2. invalid username with valid password
3. valid username with invalid password
4. valid login
5. logout verification
6. deactivated user verification
7. locked user verification after repeated invalid attempts

These tests call page methods and then use TestNG Assert.assertEquals() to validate the expected result.

STEP 6: SCREENSHOTS AND REPORTS ARE GENERATED

After each test:

* ‘BaseTest.tearDown()‘ checks whether a screenshot should be captured
* ‘ScreenshotUtils.java‘ saves screenshots under ‘test-output/screenshots/pass‘ or ‘test-output/screenshots/fail‘
* ‘HtmlSummaryReporter.java‘ creates ‘test-output/html-summary-report.html‘

This gives both visual evidence and a summary report for execution results.

6. WHEN TO USE THIS PROJECT

Use this project:

* during daily regression runs
* before releases
* after bug fixes in authentication flows
* in CI/CD pipelines for smoke checks
* when you need repeatable UI validation with screenshots

Do not rely on UI tests alone when:

* backend APIs can cover the same check faster
* the feature is still changing too frequently
* the application under test is unstable and causes false failures

A healthy approach is:

* API tests for business rules
* UI tests for user flow validation

7. HOW TO RUN THE PROJECT

PREREQUISITES

Install and configure:

* Java 21
* Maven
* a supported browser: Chrome, Firefox, or Edge
* the Reetrak application URL must be reachable

STANDARD EXECUTION

Run from the project root:

powershell
mvn clean test

RUN ON A SPECIFIC BROWSER

powershell
mvn clean test -Dbrowser=chrome
mvn clean test -Dbrowser=firefox
mvn clean test -Dbrowser=edge

RUN WITH A DIFFERENT URL

powershell
mvn clean test -Dbase.url=http://your-server/path

OVERRIDE TEST DATA FROM COMMAND LINE

powershell
mvn clean test -Dtestdata.username=MyUser -Dtestdata.password=MyPassword

8. HOW TO READ AND USE THE MAIN FILES

‘POM.XML‘

Use this file to:

* add or update dependencies
* update Java version
* control Maven plugins
* define Surefire execution behavior

Current notable dependencies:

* ‘selenium-java‘
* ‘testng‘
* ‘webdrivermanager‘
* ‘commons-io‘
* ‘slf4j-simple‘

‘TESTNG.XML‘

Use this file to:

* control which test classes run
* register listeners
* create multiple suites or groups later

Right now it runs LoginLogoutTest and registers HtmlSummaryReporter.

‘CONFIG.PROPERTIES‘

Use this file for:

* browser selection
* base URL
* test credentials and test data

This is the easiest place to change environment-specific values for local runs.

‘BASETEST.JAVA‘

Use this class when creating new tests. New test classes should usually extend BaseTest.

It provides:

* initialized ‘driver‘
* ‘baseUrl‘
* ‘testData‘
* setup and teardown lifecycle
* screenshot integration

‘BASEPAGE.JAVA‘

Use this as the parent class for page objects. Add common actions here when multiple pages need the same behavior.

‘LOGINPAGE.JAVA‘

Use this as the model for creating future page objects.

It already demonstrates:

* element location with ‘@FindBy‘
* action methods
* validation getter methods
* workflow methods like ‘loginAs()‘

9. HOW TO ADD NEW TEST CASES

When adding a new test case, follow this pattern:

1. Identify the user flow to validate.
2. Check whether the required page object already has the needed methods.
3. Add new locators or actions in the relevant page class if needed.
4. Create a new test method in the appropriate test class.
5. Use assertions for expected outcomes.
6. Run the suite and verify the report and screenshots.

EXAMPLE PROCESS

Suppose you want to test "password reset opens correctly":

1. Add locator and method in ‘LoginPage.java‘:

java
```
@FindBy(xpath = "//a[text()=’Forgot Password?’]")
private WebElement forgotPasswordLink;

public void openForgotPassword() {
    click(forgotPasswordLink);
}
```

2. Add a test method in ‘LoginLogoutTest.java‘ or a separate dedicated class:

java
```
@Test
public void forgotPasswordPageShouldOpen() {
    LoginPage loginPage = new LoginPage(driver);
    loginPage.open(baseUrl);
    loginPage.openForgotPassword();
    Assert.assertTrue(driver.getCurrentUrl().contains("forgot-password"));
}
```

BEST PRACTICE

* keep assertions in test classes
* keep browser interaction in page classes
* keep reusable setup in ‘BaseTest‘

10. HOW TO ADD A NEW PAGE OBJECT

Create a new class under:

text
src/main/java/com/reetrak/automation/pages/

Example:

java
```
package com.reetrak.automation.pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class DashboardPage extends BasePage {

    @FindBy(xpath = "//span[contains(normalize-space(), 'Dashboard')]")
    private WebElement dashboardTitle;

    public DashboardPage(WebDriver driver) {
        super(driver);
    }

    public String getDashboardTitle() {
        return getText(dashboardTitle);
    }
}
```

Use a new page object when:

* the application moves to a new screen after an action
* a page contains enough elements and methods that it should be separated for clarity
* multiple tests reuse the same screen behavior

11. HOW TO ADD A NEW TEST CLASS

Create a class under:

text
src/test/java/com/reetrak/automation/tests/

Template:

java

```
package com.reetrak.automation.tests;

import com.reetrak.automation.base.BaseTest;
import com.reetrak.automation.pages.LoginPage;
import org.testng.Assert;
import org.testng.annotations.Test;

public class ExampleTest extends BaseTest {

    @Test
    public void sampleTest() {
        LoginPage loginPage = new LoginPage(driver);
        loginPage.open(baseUrl);
        Assert.assertTrue(true);
    }
}
```

Then register the class in testng.xml if the suite should run it.

12. HOW TO UPDATE EXISTING TESTS

Update tests when:

* UI locators change
* expected messages change
* workflow steps change
* the application introduces new validation rules

UPDATE FLOW

1. Check whether the locator failed or the business expectation changed.
2. Update page locators in the page class, not in the test class.
3. Update assertions in tests if the expected output changed.
4. Run the suite again.
5. Review screenshots and HTML report.

EXAMPLE UPDATES

* If a button ‘id‘ changes, update the ‘@FindBy‘ in the page object.
* If a validation message changes, update the getter target or assertion text.
* If a login step now includes OTP, create new page methods or a new page object.

13. HOW TO UPDATE CONFIGURATION

Open:

text
src/test/resources/config.properties

Update values such as:

properties
browser=edge
base.url=http://new-server/app
testdata.username=NewAdmin
testdata.password=NewPassword

Use file-based changes when:

* you want a stable local default setup

Use command-line overrides when:

* you want temporary environment-specific execution
* you are running in CI/CD
* you do not want to store credentials in files

14. HOW TO UPDATE DEPENDENCIES

Open pom.xml and update the version values in <properties>.

Examples already present:

* ‘selenium.version‘
* ‘testng.version‘
* ‘webdrivermanager.version‘
* ‘commons-io.version‘
* ‘slf4j.version‘
* ‘surefire.version‘

After updating:

powershell
mvn clean test

Update dependencies when:

* browser compatibility issues appear
* security fixes are needed
* you want newer Selenium or TestNG features

Be careful:

* large Selenium upgrades can change browser behavior
* plugin upgrades can affect build execution
* browser-driver compatibility should be re-tested after changes

15. HOW REPORTING WORKS

SCREENSHOTS

ScreenshotUtils.java captures screenshots after tests and stores them under:

text
test-output/screenshots/pass
test-output/screenshots/fail

HTML SUMMARY REPORT

HtmlSummaryReporter.java generates:

text
test-output/html-summary-report.html

It includes:

* total test count
* passed count
* failed count
* skipped count
* duration
* screenshot links
* error messages

Use this report after every run to quickly see what happened.

16. CURRENT STRENGTHS OF THE PROJECT

* clean use of Page Object Model
* centralized driver setup
* reusable configuration handling

* custom HTML reporting
* screenshot evidence
* support for multiple browsers
* reasonable test lifecycle organization using ‘BaseTest‘

17. CURRENT LIMITATIONS AND IMPROVEMENT IDEAS

These are important if you continue growing the framework.

‘THREAD.SLEEP()‘ IS USED IN TESTS AND PAGE METHODS

This makes tests slower and more brittle. Prefer explicit waits through WaitUtils.

LOGIN AND LOGOUT ARE STILL TIGHTLY COUPLED TO ONE PAGE CLASS

As the application grows, create dedicated page classes such as:

* ‘DashboardPage‘
* ‘ProfileMenuComponent‘
* ‘ForgotPasswordPage‘

CREDENTIALS ARE STORED IN A PLAIN PROPERTIES FILE

This is acceptable for local testing, but sensitive data should eventually move to:

* environment variables
* CI secret storage
* encrypted vault solutions

ASSERTIONS USE EXACT UI STRINGS

That is okay for now, but minor wording changes can break tests. Where suitable, combine message validation with presence or workflow validation.

ONE SUITE FILE CURRENTLY RUNS ONE TEST CLASS

As the framework grows, organize suites by:

* smoke
* regression
* sanity
* module-specific execution

18. RECOMMENDED USAGE PATTERN FOR TEAM MEMBERS

FOR A TESTER

* update ‘config.properties‘
* run ‘mvn clean test‘
* open the HTML report
* inspect screenshots for failed scenarios

FOR AN AUTOMATION ENGINEER

* add or update page objects
* add reusable methods in ‘BasePage‘
* add new tests in the test package
* improve waits and reporting

FOR CI/CD

* use command-line system properties for environment selection
* archive ‘test-output‘ as a build artifact
* publish the HTML report if your pipeline supports it

19. SUGGESTED FUTURE ENHANCEMENTS

* replace ‘Thread.sleep()‘ with explicit waits
* add log utility support
* support headless browser execution
* introduce environment profiles such as dev, qa, uat
* add data-driven testing with ‘@DataProvider‘
* add retry logic only where justified
* add Allure or ExtentReports if richer reporting is needed
* add parallel-safe test design before enabling parallel execution

20. QUICK START FOR ADDING AND UPDATING

TO ADD A NEW TEST

1. Extend ‘BaseTest‘
2. Reuse or create a page object
3. Add actions in the page object
4. Add assertions in the test
5. Register the class in ‘testng.xml‘
6. Run Maven tests

TO UPDATE AN EXISTING FLOW

1. Update locator or method in the page object
2. Update test assertion if behavior changed
3. Update config if credentials or URL changed

4. Run tests and review report

TO UPDATE DEPENDENCIES

1. Modify version properties in ‘pom.xml‘
2. Run ‘mvn clean test‘
3. fix any compatibility issues

21. IMPORTANT FILES REFERENCE

* ‘pom.xml‘: build, dependency, and plugin management
* ‘testng.xml‘: suite definition and listener registration
* ‘src/test/resources/config.properties‘: runtime configuration and test data
* ‘src/test/java/com/reetrak/automation/base/BaseTest.java‘: test lifecycle
* ‘src/main/java/com/reetrak/automation/factory/DriverFactory.java‘: browser setup
* ‘src/main/java/com/reetrak/automation/pages/BasePage.java‘: shared page actions
* ‘src/main/java/com/reetrak/automation/pages/LoginPage.java‘: login/logout page actions and validations
* ‘src/main/java/com/reetrak/automation/utilities/WaitUtils.java‘: explicit waits
* ‘src/main/java/com/reetrak/automation/utilities/ScreenshotUtils.java‘: screenshot capture
* ‘src/test/java/com/reetrak/automation/reports/HtmlSummaryReporter.java‘: HTML reporting
* ‘src/test/java/com/reetrak/automation/tests/LoginLogoutTest.java‘: actual login/logout scenarios

22. FINAL SUMMARY

This project is a maintainable Selenium TestNG UI automation framework built for Reetrak login-related verification. Its main purpose is to automate authentication checks, reduce manual regression effort, and provide execution proof through reports and screenshots.

Use it when you need reliable browser-based validation. Add to it by creating new page objects and tests. Update it by changing locators, assertions, configuration, and dependency versions in the correct layers. If you keep page actions, test assertions, configuration, and reporting well separated, this framework will remain easy to extend over time.